

Lab 1: Running Synthetic Traffic on Garnet

王立明
2024011338

Abstract

Garnet 3.0 (gem5 v23.0.0.1, Garnet standalone, NULL ISA) is exercised with uniform-random single-flit traffic on a 64-node 8×8 Mesh. Task 1 changes the global tick frequency from 1ps to 2GHz; because the traffic generator compares `--sim-cycles` against *ticks*, this turns a degenerate 20-cycle run (2 packets delivered) into the intended 10000-cycle run (3162 packets). Task 2 adds explicit units to all Garnet statistics and a new `reception_rate` metric, verified analytically: $3162/(64 \times 10000) = 0.004941$ packets/node/cycle. Task 3 answers the five questions on parameters, time units, and the modules that generate, buffer, and gate packets.

1 Setup

```
scons build/NULL/gem5.opt PROTOCOL=Garnet_standalone -j$(nproc)

./build/NULL/gem5.opt configs/example/garnet_synth_traffic.py \
  --network=garnet --num-cpus=64 --num-dirs=64 \
  --topology=Mesh_XY --mesh-rows=8 --inj-vnet=0 \
  --synthetic=uniform_random --sim-cycles=10000 --injectionrate=0.01
```

2 Task 1: Global frequency to 2 GHz

The script ends with `m5.ticks.setGlobalFrequency("1ps")` (`garnet_synth_traffic.py:176`), which sets the resolution of the global event queue: 1ps means 10^{12} ticks/s, the replacement 2GHz means 2×10^9 ticks/s. The Ruby/Garnet clock is 2 GHz (`--ruby-clock`), so one network cycle is 500 ticks before the change and exactly 1 tick after it. The change matters because the tester stops on `if (curTick() >= simCycles)` (`GarnetSyntheticTraffic.cc:175`) — a *tick* count. At 1ps, “10000 cycles” expires after $10000/500 = 20$ real network cycles; at 2GHz the full 10000 cycles are simulated.

Table 1: `network_stats.txt` before and after the change.

Metric	1ps (before)	2GHz (after)
network cycles actually simulated	20	10000
packets_injected	5	3165
packets_received	2	3162
average_packet_queueing_latency	1000	2
average_packet_network_latency	3000	13.557559
average_packet_latency	4000	15.557559
average_hops	1.500000	5.269450
reception_rate	0.001563	0.004941

Two checks confirm the interpretation. The 1ps latencies 1000/3000 are ticks; divided by 500 ticks/cycle they are 2/6 cycles — plausible for the only two nearest-neighbour packets that

can arrive in 20 cycles. And the 10000-cycle `average_hops` = 5.27 matches the mean Manhattan distance of uniform-random traffic on an 8×8 mesh, $2 \cdot \frac{8^2-1}{3 \cdot 8} = 5.25$ (probe runs with a fixed sender/destination pair confirm the counter measures router-to-router distance: 0 for self-traffic, 1 for a neighbour, 14 corner-to-corner). The three missing packets are in flight at termination.

3 Task 2: Units and Reception Rate

Every statistic in `GarnetNetwork::regStats()` now carries a unit from `src/base/stats/units.hh` via `.unit()`:

Table 2: Units assigned in `GarnetNetwork.cc`.

Statistic	Unit
packets/flits injected and received; total hops; link counters	Count
accumulated packet/flit latencies	Tick
<code>average*_latency</code> formulas (10)	Tick/Count
<code>average_hops</code>	Count/Count
<code>avg_link_utilization</code> , <code>avg_vc_load</code>	Ratio
<code>sim_cycles</code> (new)	Cycle
<code>reception_rate</code> (new)	Count/Cycle

The latency counters accumulate *ticks*, so their unit is `Tick`; after Task 1 one tick equals one Ruby cycle, so the printed values read directly as cycles.

Reception rate is total packets received/(num-cpus $\times$ sim-cycles). The numerator exists as `m_packets_received` and the node count as `m_routers.size()`; the measurement window is sampled where `collateStats()` already computes it:

```
// GarnetNetwork.cc, collateStats(): record the measurement window
m_sim_cycles = double(curCycle() - rs->getStartCycle());

// GarnetNetwork.cc, regStats(): register the derived metric
m_reception_rate = sum(m_packets_received) /
    (m_sim_cycles * (double)m_routers.size());
```

A `statistics::Formula` keeps the metric consistent under stat resets; `collateStats()` runs as a dump callback, i.e. before formulas are evaluated. Verification: $3162/(64 \times 10000) = 0.004941$ ✓. Both the injected and the accepted rate are half the offered 0.01 because the generators tick in the 1 GHz system clock domain while Garnet cycles at 2 GHz — one injection opportunity every second network cycle. Lab 2 accounts for this factor of two throughout.

4 Task 3: Questions

4.1 Input parameters of `garnet_synth_traffic.py`

`-h` shows four option layers, merged by `Options.addNoISAOptions()`, `Ruby.define_options()` and the script itself.

Table 3: Traffic options — defined in `configs/example/garnet_synth_traffic.py`.

Option	Default	Usage
<code>--synthetic</code>	<code>uniform_random</code>	pattern: <code>uniform_random</code> , <code>tornado</code> , <code>bit_complement</code> , <code>bit_reverse</code> , <code>bit_rotation</code> , <code>neighbor</code> , <code>shuffle</code> , <code>transpose</code>
<code>-i, --injectionrate</code>	0.1	offered load, packets per node per cycle
<code>--precision</code>	3	digits used to quantise the rate
<code>--sim-cycles</code>	1000	run length
<code>--num-packets-max</code>	-1	stop injecting after N packets
<code>--single-sender-id</code>	-1	restrict to one source
<code>--single-dest-id</code>	-1	restrict to one destination
<code>--inj-vnet</code>	-1	vnet 0/1 (1-flit) or 2 (5-flit); -1 random

Table 4: Network options — defined in `configs/network/Network.py`.

Option	Default	Usage
<code>--network</code>	<code>simple</code>	<code>simple</code> or <code>garnet</code>
<code>--topology</code>	<code>Crossbar</code>	class in <code>configs/topologies</code>
<code>--mesh-rows</code>	0	rows of a 2-D topology
<code>--router-latency</code>	1	router pipeline depth (cycles)
<code>--link-latency</code>	1	per-link latency (cycles)
<code>--link-width-bits</code>	128	link width; flit size = width/8 bytes
<code>--vcs-per-vnet</code>	4	VCs per virtual network
<code>--routing-algorithm</code>	0	0 table, 1 XY, 2 custom
<code>--garnet-deadlock-threshold</code>	50000	deadlock detection

The remaining layers: system-level options (`--num-cpus`, `--sys-clock` 1 GHz, memory/-caches) in `configs/common/Options.py`; Ruby options (`--ruby-clock` 2 GHz, `--num-dirs`, ...) in `configs/ruby/Ruby.py`. These are the *configuration* defaults; the `SimObject` parameter defaults live separately in `src/mem/ruby/network/garnet/GarnetNetwork.py` — notably `buffers_per_data_vc = 4` and `buffers_per_ctrl_vc = 1`, which have no command-line option at all.

4.2 Units of `sim-cycles`, `router-latency`, `link-latency`; Tick vs Cycle

`sim-cycles` is nominally cycles but implemented as *ticks* (`curTick() >= simCycles`); the two coincide only after Task 1’s change. `router-latency` and `link-latency` are both in *cycles* of the Ruby clock domain (pipeline stages per router, traversal cycles per link). A *Tick* is `gem5`’s global event-queue time unit; a *Cycle* belongs to one clock domain: $\text{ticks per cycle} = \text{simFreq}/f_{\text{domain}}$. `ClockedObject` converts via `clockEdge()`, `curCycle()`, `cyclesToTicks()`.

4.3 Unit of `injectionrate`

Packets per node per cycle — the probability, in $(0, 1]$, that a node injects a packet in a given generator cycle (if `(trySending < injRate*injRange)`, `GarnetSyntheticTraffic.cc:155`). Since a node can accept at most one flit per cycle, 1.0 is the theoretical maximum.

4.4 Where `GarnetNetworkInterface` and `GarnetRouter` are defined

Both `SimObjects` are declared in `src/mem/ruby/network/garnet/GarnetNetwork.py` (lines 56 and 73), pointing at the C++ classes `gem5::ruby::garnet::NetworkInterface` (`NetworkInterface.hh/.cc`) and `gem5::ruby::garnet::Router` (`Router.hh/.cc`) in the same directory. The generated `GarnetNetworkInterfaceParams` / `GarnetRouterParams` structs tie the Python names to the C++ implementations.

4.5 Which modules generate, buffer, and gate packets

Generation/injection: `GarnetSyntheticTraffic.cc` decides each cycle whether to inject and picks the destination; `NetworkInterface::flitIsSizeMessage()` then converts the message into flits (`HEAD_/BODY_/TAIL_`, or `HEAD_TAIL_` if single-flit), allocates an output VC, and puts them on the injection link.

Buffering: inside routers, exclusively in the input buffers — `InputUnit` holds one `VirtualChannel` (a `flitBuffer` of depth `buffers_per_*_vc`) per VC per port; the NI additionally buffers at the edges (`m_ni_out_vcs`, `outFlitQueue`) and the crossbar holds the flit in traversal. Garnet routers have no output buffering.

Downstream decision: `RoutingUnit` picks the direction (`outportCompute()`); `OutputUnit` tracks downstream space (`has_free_vc()`, `has_credit()`, credit counters in `OutVcState`); `SwitchAllocator::s` makes the go/no-go call — a free output VC (head flits) or a credit on the held VC (body/tail), plus an ordering check on ordered vnets.

5 Conclusion

The frequency fix makes one tick equal one Ruby cycle, so the requested 10000 cycles are actually simulated; the before/after table quantifies the degenerate behaviour of the stock setting. The unit annotations make the statistics self-describing, and `reception_rate` is verified against the raw counters. The source walk-through locates each stage of a packet’s life: generated in the tester, flitisized in the NI, buffered per-VC in each router’s `InputUnit`, and advanced only when `RoutingUnit`, `OutputUnit` and `SwitchAllocator` agree.